Account
Dashboard
Courses
Groups
Calendar
Inbox
History
Help

Summer 2026

Home
Modules
Announcements
Discussions
Grades 1

Mini-Project

Due: Mon Aug 10, 2026 11:59pm

48 Points Possible

Attempt 1

In Progress
NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed
Available: Jun 29, 2026 11:59pm until Aug 16, 2026 11:59pm

Details

In this Mini Project (MP), you have the choice to practice applying your **Web & HTTP** and **Socket Programming** knowledge or **Reliable Data Transfer** knowledge to build a reliable transport protocol on top of an unreliable channel.

Estimated Required Time: Total of 8-15 hours per person*
*Estimate based on the level of course practice with Python socket programming, and how detailed the project and reporting are performed.

The second option is considered a little bit more complex, therefore it has an opportunity to gain bonus points (in case of successful implementation of the bonus parts).

Step One: Choose whether you want to implement option (1) or option (2) for your project.

Option (1): Web & Web Proxy Servers

Step Two: Determine Requirements (10 points)

At this step, you need to find out the details needed to implement your web server. We need you to implement a simple web server that is able to process and respond to HTTP requests. Your web server should be able to respond to requests with the following messages for the relevant methods to the client. Therefore, for each item you need to decide what are the requirements in the request, what method is used, and what part of the message is causing the specified status code in the response (2 points for each item):

Code	Message
200	OK
304	Not Modified
403	Forbidden
404	Not Found
505	HTTP Version Not Supported

Please feel free to consult RFC 7231, Section 6 for details of the Status Codes.
The deliverable of this step is writing down the specifications of the logic for the generation of each of these status codes (1 point each), and the HTTP request message you will use to test it (1 point each).

Step Three: Build Your Minimal Web Server & Test (10 points)

(a) You can start this step once you have seen socket programming at the end of Module (2). In this part of your MP, using what you learned about socket programming and HTTP protocol, we require you to create your simple web server. Note that you are not allowed to use the Python HTTP module for this project.

Note that HTTP protocol is a request/response protocol. Therefore, implementing a Web Server means writing code for a server that can receive HTTP messages sent to a known port number that is listening to client requests, processing those requests, and responding to them.

(b) To test your web server for the correct working scenario, copy test.html in the main directory of your web server. Then find out the IP address of your machine (or use the local host address), and the port used in the code for the web server and type the following in your web browser (2 points):

```
http://IP_ADDRESS:PORT/test.html
```

(c) Now you need to test your web server for generating the status codes provided in step one. You can use **curl**, **telnet**, or your **browser** for each of these steps. You can also edit your test file, or send the request properly to test your implementation for different message scenarios. Print screen, or cut and paste the output and document your test procedures. (8 points)

Step Four: Performance (20 points)

(a) Think about a web proxy server. What is different in request handling in a proxy server and a web server that hosts your files? Write down the detailed specifications you come up with for a minimal proxy server only using the knowledge you have from module (2) slides 29-34 (and consulting RFCs), and implement them (10 points).

(b) Decide the test procedure to show the working of your proxy server. You need to describe and find ways to test your server-side functionality. Print screen, or cut and past output and document your test procedures to show. (5 points)

(c) Is your Web Server a single-thread server or a multi-thread one? If it is a single-thread server, expand it to respond to parallel requests in different threads. Explain in the report document why your web server is a multi-thread server and how that impacts performance. (5 points)

Step Five: Prevent Head of Line Blocking (8 pts)

Make changes to your server to avoid the HOL problem using frames (5 points), and explain what you have done (3 points).
Note: Please check Module (2) Slide 38 for more details.

Option (2): Your Pipelined Reliable Transfer Protocol

Step Two: Design (15 points) & Step Three: Implement your protocol (15 points):

TCP is the reliable transport protocol of the Internet. We have studied and observed the TCP protocol behavior in IS(4). In this step of your project, we want you to implement your flavour of connection-oriented, reliable, pipelined protocol which includes flow control and congestion control mechanisms as well. To do this, each group needs to specify their mechanisms in writing, decide the socket type (which does not provide these services by default), and define and implement the defined Pipelined Reliable Transfer protocol.

Step Four: Analyze Traffic (18 points):

In this step of the project, you need to analyze the traffic of your transport protocol using Wireshark (or a similar capture of your packets). Through this traffic analysis, you need to prove that your protocol is connection-oriented, has acceptable performance, and implements flow control and congestion control mechanisms.

OK, everything by far seems straightforward. So, where is the complexity?

Remember the discussion we had using udt_send(), rdt_send(), and rdt_receive() functions? Module (3), Slide 19!
When you use TCP/UDP sockets and send and receive packets, even though you may use the proper socket that behaves the way is expected, you may not be able to test your protocols in a controlled environment for testing (e.g., you run in a computer locally and so you do not have any loss or bit flips in your best effort channel). So, to enable proper testing of your design, simulation of loss and errors is required.

How do you do that? There are multiple possible ways you can approach it, all involving using a random function and using it for dropping packets, the main question is: where? So, that is one part of the problem-solving pieces we expect in this Mini-Project. You can also bring your questions about it to the classroom.

Step Five: Bonus: Prove Fairness (5 points):
Test your protocol with other traffic (2 points). You can choose to use network simulators, run your protocol across the network, or bring up your virtual nodes to test. Is your protocol fair? Can you prove it? (3 points)

Deliverables

Specify your choice of the mini project, then for steps two to five, please make a zip file including:

- Your code (.py)
- Your (if any) modified test files (.html), and/or capture and analysis (e.g., figure) files
- A report (.pdf) file including specifications of your protocol, your test procedures, and an explanation of your results showing the desired working of your protocol (e.g., status code generation logic, proxy requirements, and your test procedures including the request message details for all steps, and print screens)

and submit it to this activity in Canvas. One submission will be enough for each group (should automatically show for both group members).

Resources

[1] RFC 7231 (https://tools.ietf.org/html/rfc7231.html), Section 6
[2] RFC 7540 (https://tools.ietf.org/html/rfc7540)
[3] https://docs.python.org/3/howto/sockets.html
Also covered in IS(3): UDP, IS(4): TCP, and IS(6): RAW
[4] https://docs.python.org/3/library/threading.html
[5] Web Proxy (Not a Standard - architecture in section 3.2.2 is intended): https://www.ietf.org/rfc/rfc3040.txt

Note (1): Your project needs to be your work. Reuse of similar project codes and/or other similar student projects is not allowed. If you are using parts of a resource, you need to ensure usage permissions and cite the work you are using.

Note (2): You are not allowed to use the Python HTTP modules/libraries. You are allowed to consult IETF RFCs for everything.

Note (3): The steps could be additive implementation on the same server Python file. However, if you are not sure and afraid that your changes will impact your previous step's output, you can mention that in your report and include multiple Python files, one for each step.

Note (4): The MP team has 2 points, MP choice (1) has 48 points, and MP choice (2) has 48 points and 5 bonus points. Each of the choices sum up to 50 points which will be 15% of your total grade. The bonus point on the choice (2) will be added to this value (different from the 2% bonus of class activities).

! Keep in mind, this submission will count for everyone in your MP Team group.

Choose a submission type

Upload
More

Drag a file here, or
Choose a file to upload
File permitted: ZIP

or

Canvas Files